Birla Institute of Technology & Science, Pilani Hyderabad Campus

CS F372: Operating Systems (Assignment No:3)

Assigned: 02/11/2026 **Date of Submission: 13/11/2026**
Total Marks: 16 **Weightage: 8%**

General Instructions:

- This assignment can be done in groups of **maximum 4** students.
- All your programs must run on a **Ubuntu 24.X virtual machine on Oracle Virtual Box**.
- Recommended allocation: 4 cores, 50 GB virtual disk, 4GB RAM or more on VM.
- Submit all source files and executables along with a readme that has group details (Names and ID numbers of all members of the group.)
- Put all your deliverables in a zip/tar file and submit through the course webpage (assignment section). One submission per group is sufficient.
- Every assignment will have a demo and viva, the schedule of which will be intimated later through a separate notice.

For any assignment related queries, you may contact I/C or the following teaching assistants:

- Arpit Kudre <f20230158@hyderabad.bits-pilani.ac.in>
- Parth Munjal <f20230058@hyderabad.bits-pilani.ac.in>

Q1. The Mission Control Telemetry Vault: [5 Marks]

Problem Overview:
You are programming the access-control software for the telemetry vault at a Mars ground station. The vault holds one shared telemetry record: a version number and 8 sensor fields. Six analyst threads (Analysts 0 through 5) read the record, and two uplink threads (Uplinks 0 and 1) overwrite it. An archiver thread periodically takes a full snapshot of the vault. Because the record must never be seen half-updated, access follows these rules:

- Up to 3 analysts may read at the same time.
- An uplink needs exclusive access (no analysts, no other uplink, no archiver).
- The archiver also needs exclusive access.

Thread Behaviour:

- **Analyst:** sleeps (idle) for a random 1 to 4 seconds, requests read access, then reads for a random 1 to 2 seconds. While reading, it verifies that all 8 fields hold the same value as the version number and prints CONSISTENT or CORRUPT.
- **Uplink:** sleeps (waiting for a satellite pass) for a random 2 to 5 seconds, requests write access, then writes for a random 1 to 2 seconds. A write increments the version and sets all 8 fields to the new version, one field at a time with a small `usleep()` between fields so that any lack of mutual exclusion becomes visible.
- **Archiver:** every 12 seconds it requests exclusive access and holds it for 2 seconds. Once the archiver has announced its request, no new analyst or uplink may enter until the archiver has finished. Threads already inside are allowed to finish.

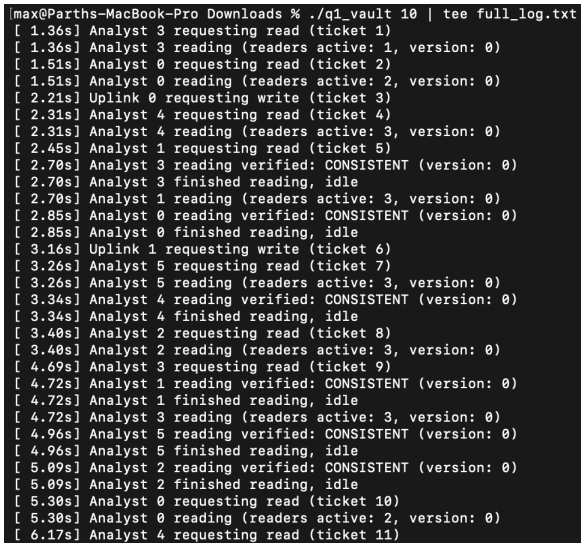
Aging and Deadline Rules: Every request receives a ticket number from a global arrival counter.

- A request that has waited more than 4 seconds becomes URGENT. Urgent requests are served before non-urgent ones, in ticket order among themselves. A non-urgent request must not be admitted ahead of an older waiting request of the other type.
- A request that has waited more than 10 seconds must be abandoned. The thread logs a TIMEOUT, removes its request so that it never blocks anyone else, backs off for 2 seconds and then retries with a new ticket. Use `pthread_cond_timedwait()` with a condition variable configured for `CLOCK_MONOTONIC`.
- The program runs for 60 seconds and then shuts down cleanly (no thread may be left blocked). It prints a summary table with, for each thread, the number of accesses completed, the maximum wait time and the number of timeouts.

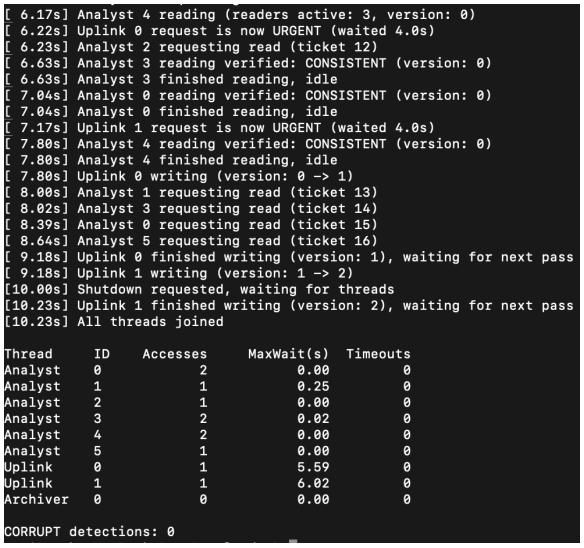
Technical Constraints:

- You must strictly use `pthread_mutex_t` and `pthread_cond_t`. Semaphores and `pthread_rwlock_t` are not allowed.
- Use separate condition variables for analysts, uplinks and the archiver. In your readme, justify each place where you used `pthread_cond_signal` versus `pthread_cond_broadcast`.
- Your solution must be free of deadlocks and starvation, and the CORRUPT message must never appear.
- You must print whenever a thread's state changes (idle, waiting, urgent, reading/writing, timeout, finished).

Sample Output: To verify your solution, the program must print a log of events to the console. Each line should include the thread type, its ID, and the current version and ticket where relevant:



```
max@Parths-MacBook-Pro Downloads % ./q1_vault 10 | tee full_log.txt
[ 1.36s] Analyst 3 requesting read (ticket 1)
[ 1.36s] Analyst 3 reading (readers active: 1, version: 0)
[ 1.51s] Analyst 0 requesting read (ticket 2)
[ 1.51s] Analyst 0 reading (readers active: 2, version: 0)
[ 2.21s] Uplink 0 requesting write (ticket 3)
[ 2.31s] Analyst 4 requesting read (ticket 4)
[ 2.31s] Analyst 4 reading (readers active: 3, version: 0)
[ 2.45s] Analyst 1 requesting read (ticket 5)
[ 2.70s] Analyst 3 reading verified: CONSISTENT (version: 0)
[ 2.70s] Analyst 3 finished reading, idle
[ 2.70s] Analyst 1 reading (readers active: 3, version: 0)
[ 2.85s] Analyst 0 reading verified: CONSISTENT (version: 0)
[ 2.85s] Analyst 0 finished reading, idle
[ 3.16s] Uplink 1 requesting write (ticket 6)
[ 3.26s] Analyst 5 requesting read (ticket 7)
[ 3.26s] Analyst 5 reading (readers active: 3, version: 0)
[ 3.34s] Analyst 4 reading verified: CONSISTENT (version: 0)
[ 3.34s] Analyst 4 finished reading, idle
[ 3.40s] Analyst 2 requesting read (ticket 8)
[ 3.40s] Analyst 2 reading (readers active: 3, version: 0)
[ 4.69s] Analyst 3 requesting read (ticket 9)
[ 4.72s] Analyst 1 reading verified: CONSISTENT (version: 0)
[ 4.72s] Analyst 1 finished reading, idle
[ 4.72s] Analyst 3 reading (readers active: 3, version: 0)
[ 4.96s] Analyst 5 reading verified: CONSISTENT (version: 0)
[ 4.96s] Analyst 5 finished reading, idle
[ 5.09s] Analyst 2 reading verified: CONSISTENT (version: 0)
[ 5.09s] Analyst 2 finished reading, idle
[ 5.30s] Analyst 0 requesting read (ticket 10)
[ 5.30s] Analyst 0 reading (readers active: 2, version: 0)
[ 6.17s] Analyst 4 requesting read (ticket 11)
```



```
[ 6.17s] Analyst 4 reading (readers active: 3, version: 0)
[ 6.22s] Uplink 0 request is now URGENT (waited 4.0s)
[ 6.23s] Analyst 2 requesting read (ticket 12)
[ 6.63s] Analyst 3 reading verified: CONSISTENT (version: 0)
[ 6.63s] Analyst 3 finished reading, idle
[ 7.04s] Analyst 0 reading verified: CONSISTENT (version: 0)
[ 7.04s] Analyst 0 finished reading, idle
[ 7.17s] Uplink 1 request is now URGENT (waited 4.0s)
[ 7.80s] Analyst 4 reading verified: CONSISTENT (version: 0)
[ 7.80s] Analyst 4 finished reading, idle
[ 7.80s] Uplink 0 writing (version: 0 -> 1)
[ 8.00s] Analyst 1 requesting read (ticket 13)
[ 8.02s] Analyst 3 requesting read (ticket 14)
[ 8.39s] Analyst 0 requesting read (ticket 15)
[ 8.64s] Analyst 5 requesting read (ticket 16)
[ 9.18s] Uplink 0 finished writing (version: 1), waiting for next pass
[ 9.18s] Uplink 1 writing (version: 1 -> 2)
[10.00s] Shutdown requested, waiting for threads
[10.23s] Uplink 1 finished writing (version: 2), waiting for next pass
[10.23s] All threads joined

Thread     ID    Accesses     MaxWait(s)  Timeouts
Analyst    0            2           0.00         0
Analyst    1            1           0.25         0
Analyst    2            1           0.00         0
Analyst    3            2           0.02         0
Analyst    4            2           0.00         0
Analyst    5            1           0.00         0
Uplink     0            1           5.59         0
Uplink     1            1           6.02         0
Archiver   0            0           0.00         0

CORRUPT detections: 0
```

(Hint: Because C does not have a built-in monitor construct, you must implement the monitor logic using Pthreads Mutexes and Condition Variables. Think carefully about which events must wake which waiters.)

Q2. Transitive priority inheritance, priority ceiling & deadlock detection. [6 Marks]

Write a C program using POSIX threads on a single CPU (pin it with `sched_setaffinity()` or `taskset -c 0`, and run as root so SCHED_FIFO works).

Part A: Create four SCHED_FIFO threads and two mutexes, M1 and M2, as follows.

- A low-priority thread L (priority 10) starts at 0 ms, locks M1 and does 600 ms of busy-loop CPU work inside it.
- A thread D (priority 20) starts at 100 ms, locks M2, then tries to lock M1. Once it acquires M1 it does 100 ms of busy-loop work and then releases both mutexes.
- A high-priority thread H (priority 30) starts at 200 ms and tries to lock M2.
- A thread X (priority 25) starts at 300 ms and does 2 s of busy-loop work without using any mutex.

H is therefore blocked on a chain (H waits for D, D waits for L). Run three versions, each 5 times, and report the average:

- default mutexes (`PTHREAD_PRIO_NONE`);
- `PTHREAD_PRIO_INHERIT` on both mutexes;
- `PTHREAD_PRIO_PROTECT` with a ceiling of 30 on both mutexes.

For every run, record two measurements for H: (i) its blocking time (the time from calling `pthread_mutex_lock` to acquiring it), and (ii) its response latency (the time from its intended release at 200 ms to acquiring M2). Before running, write down your predicted value of each for all three versions. Then present a table of predicted versus measured values, and explain any mismatch. In particular, explain why (i) and (ii) can differ for version 3. Also log a timestamped event trace (thread started, lock requested, lock acquired, lock released, finished) for one run of each version.

Sample Output: the start of the Part A event trace, cut off at H's first lock request.

```
=== PRIO_NONE ===
  event trace (PRIO_NONE, run 1):
    [    0.14 ms] L  started
    [    0.14 ms] L  lock M1 requested
    [    0.14 ms] L  lock M1 acquired
    [  100.04 ms] D  started
    [  100.04 ms] D  lock M2 requested
    [  100.04 ms] D  lock M2 acquired
    [  100.04 ms] D  lock M1 requested
    [  200.03 ms] H  started
    [  200.03 ms] H  lock M2 requested
```

Part B: Create three mutexes A, B and C and three threads, where T1 locks A, sleeps 100 ms and then locks B, T2 locks B, sleeps 100 ms and then locks C, and T3 locks C, sleeps 100 ms and then locks A. First show that the program hangs (deadlock), using a watchdog thread that maintains a wait-for graph (updated by your own lock/unlock wrapper functions) and prints the detected cycle (for example `T1 -> B -> T2 -> C -> T3 -> A -> T1`) within 1 s of the hang.

Then fix it without changing the lock order. Replace blocking locks with `pthread_mutex_timedlock()` (300 ms timeout). On timeout, a thread must release everything it holds, wait a random 50 to 250 ms, and retry. Run the fixed version 20 times and report the number of retries per run and the completion time (average and maximum). Report any run that took unusually long.

In a short note (5 to 6 lines), explain why priority inheritance in Part A cannot help in Part B, name the Coffman condition(s) your timeout-and-retry fix breaks, and explain why the fix can still fail to make progress (livelock) in theory and what you did to make that unlikely. Use `clock_gettime(CLOCK_MONOTONIC)` for all timing, and make all critical sections busy loops (not sleep, which would release the CPU and hide the inversion).

Submit the source code, the predicted versus measured table for the three mutex types, the event traces, evidence of the detected hang and of the fix, and the short explanation.

Sample Output:
The hang demo, with the watchdog printing `T1 -> B -> T2 -> C -> T3 -> A -> T1` and the exit code 2.

```
Part B: hang demo - T1: A->B, T2: B->C, T3: C->A, blocking locks
  [watchdog   111.0 ms] DEADLOCK cycle detected: T1 -> B -> T2 -> C -> T3 -> A -> T1
  [watchdog] threads are blocked forever (no timeout in this version); exiting with code 2
```

The first run lines of the fixed version, trimmed to the first run

```
  [watchdog   100.8 ms] DEADLOCK cycle detected: T1 -> B -> T2 -> C -> T3 -> A -> T1
run  1: completion   753.1 ms   retries 2 (T1 1, T2 1, T3 0)   deadlock cycles seen by watchdog: 1
  [watchdog   100.8 ms] DEADLOCK cycle detected: T1 -> B -> T2 -> C -> T3 -> A -> T1
run  2: completion   747.5 ms   retries 2 (T1 1, T2 1, T3 0)   deadlock cycles seen by watchdog: 1
...
```

Q3. Memory pressure, locality, and swap performance [5 Marks]

Write a C program to investigate how memory allocation size, access locality, and swap storage affect performance.

Setup

Use an Ubuntu 24.04 LTS VirtualBox VM with one virtual CPU and 2 GiB RAM. Compare swap on:

- A virtual disk backed by the host's internal SSD.
- An external USB device: a pen drive, external SSD, or external HDD.

Configure 4 GiB of swap on each backing, with only the backing being tested active during a trial. Keep VM settings unchanged between tests, other than the swap backing. Disable zswap and do not use zram for these experiments. Close other guest applications and minimize host activity. Make swap changes inside the virtual machine.

Program

Allocate memory in page-aligned 1 MiB chunks, checking allocation results. Obtain the system page size with `sysconf(_SC_PAGESIZE)` and initialize a known byte at the beginning of every page in every chunk. For example, for globally numbered pages starting at zero, use `(page_number % 256)`.

These are the two patterns for one-byte lookups:

- Uniform random: select a page uniformly from the entire allocation.
- Hot region: use the first 10% of pages as a fixed hot region. For each lookup, choose the hot region with 90% probability and the remaining pages with 10% probability, then select a page uniformly within that region.

Read the initialized byte from each selected page. Lookups may repeat. Use a pseudo-random generator with your chosen seed and accumulate the lookup values in a checksum for each pass while ensuring that the compiler cannot optimize the reads away.

Trials and measurements

Part A: Find the memory-pressure transition. Using internal-SSD swap, test 512, 1024, 1536, 2048, and 2560 MiB with both patterns. If an additional size is chosen in Part B below, also test it here with both patterns.

Part B: Compare storage backings. Test 512, 1024, and 1536 MiB with both patterns using USB swap. If 1536 MiB takes an impractically long time or cannot complete, you must choose an additional allocation size yourself between 1024 and 1536 MiB, trying intermediate values in small steps. Choose a size at which a completed uniform trial shows non-zero major page faults and swap-ins during at least one access pass, rather than only during initialization. Test the additional size with both patterns and both swap backings, using the same measurement procedure, including all five passes. Reuse Part A's internal-SSD results where available.

For each trial, start a fresh process, allocate and initialize once, then perform five successive passes of 10,000 lookups before freeing the memory. Keep the allocation and hot region unchanged between passes. Continue the random generator between passes so each pass uses fresh references. Generate lookup indices before timing each access pass. Do not run trials concurrently.

Use `clock_gettime(CLOCK_MONOTONIC)` to time these phases separately:

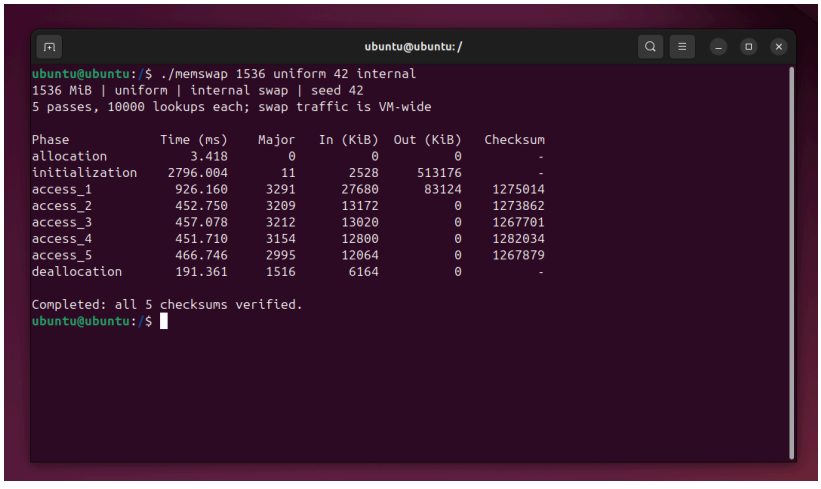
- Allocate all chunks.
- Initialize every page.
- Perform each of the five access passes, timing each separately.
- Free all chunks.

Record elapsed time in milliseconds for each phase. For initialization and each access pass, also record:

- Major page-fault increments using `getrusage(RUSAGE_SELF)`.
- Swap-in and swap-out traffic in KiB using `pswpin` and `pswpout` from `/proc/vmstat`, converting their page-count increments appropriately.

Take counter readings outside the timed intervals, and keep printing and file output outside those intervals. Swap counters are VM-wide and may include background activity.

Sample Output



Findings and submission

For Part A, submit two tables and two plots (one per table) against allocation size:

- Average access time per lookup (whole-pass time divided by 10,000), showing the first and fifth passes for both patterns.
- Major page faults for the same passes and patterns.

Also plot **average access time per lookup against pass number (1–5) at 2560 MiB**, showing uniform and hot-region access as two curves on the same plot.

For Part B, provide one comparison table for the two backings at 512 and 1024 MiB, and at 1536 MiB or your additional size. Include initialization time, first- and fifth-pass access times, major faults, and swap traffic. Use matching sizes, patterns, and pass numbers.

Submit:

- **Source code**, with brief build/run instructions.
- **One results file (.txt / .pdf)** containing the recorded measurements for all the experiments in the form of tables for both Part A and B, tested sizes, and a short explanation of your findings.
- **The three Part A plots.**